INFORME DE ARQUITECTURA DE SOFTWARE: PATRÓN MVC

Asignatura: Análisis y Diseño de Sistemas II
Escenario de Grupo: Control y Registro de Acceso Vehicular en Garita

Asignatura: Análisis y Diseño de Sistemas II	**Patrón de Diseño:** Modelo - Vista - Controlador (MVC)
Proyecto General: Sistema VigilanciaUNP	**Escenario Individual:** Control y Registro en Garita

1. ¿De qué trata la aplicación y qué hace?

VigilanciaUNP es una solución integral desarrollada para el control y fiscalización del flujo vehicular en las garitas de ingreso del campus de la **Universidad Nacional de Piura (UNP)** (Puerta 1 Principal y Puerta 4 Secundaria). El sistema moderniza la labor del personal de vigilancia sustituyendo las anotaciones manuales en cuadernos por un registro digital seguro y auditado.

Funcionalidades principales del sistema:
• **Detección y Reconocimiento de Placas mediante Inteligencia Artificial:** Emplea visión artificial y OCR en tiempo real a través de la cámara del dispositivo móvil para leer la placa vehicular automáticamente o permitir su digitación manual.
• **Consulta Inmediata al Padrón Institucional:** Cruza la matrícula con la base de datos para identificar la titularidad del vehículo (estudiante, docente, personal administrativo, servicio de entrega o visitante).
• **Control de Coherencia de Permanencia (Entrada / Salida):** Verifica el último movimiento registrado; si el vehículo ya está dentro, exige salida e impide registrar entradas duplicadas sin justificación expresa.
• **Gestión de Excepciones y Auditoría:** Permite autorizar 'Reingreso Forzado' o 'Salida Forzada' con marcas de auditoría ante omisiones de registro previas, además de auditar a *Conductores Ocasionales* preservando el titular del padrón.
• **Persistencia Local en SQLite y Reportes:** Garantiza la disponibilidad ininterrumpida de los datos sin depender de internet y genera reportes de bitácora institucional.

2. Delimitación del Escenario Asignado: Control en Garita

El escenario específico trabajado abarca el **Proceso Operativo de Control y Registro en Garita**: desde la captura o ingreso de la placa, la evaluación de las restricciones de permanencia vehicular en la universidad, la confirmación de identidad del conductor y el asentamiento seguro del movimiento.

3. Aplicación del Patrón Modelo-Vista-Controlador (MVC)

Para estructurar el escenario bajo los principios de modularidad y bajo acoplamiento de la ingeniería de software, se aplicó el patrón **Modelo-Vista-Controlador (MVC)**, dividiendo el sistema en tres componentes fundamentales:

Componente	Elementos en el Proyecto	Responsabilidad Primaria
MODELO (Model)	`Entidades.kt` `AccesoVehicularModel.kt`	Contiene el estado de los datos y las reglas del negocio: validación de placa, control de permanencia (adentro/afuera), titularidad y persistencia en SQLite.
VISTA (View)	`IVigilanciaView.kt` `ConsolaGaritaView.kt`	Interfaz pasiva de usuario. Renderiza pantallas, alertas y confirmaciones sin ejecutar lógica de negocio ni consultar bases de datos.
CONTROLADOR (Controller)	`VigilanciaController.kt`	Intermediario y orquestador. Captura eventos de la vista o del lector de cámara, solicita validación al modelo y actualiza la vista con el resultado.

4. Justificación Técnica: ¿Qué hay en cada componente y por qué?

4.1. Componente: MODELO (Model) — Datos y Reglas de Negocio

¿Qué contiene?
• **Entidades del Dominio:** Vehiculo (datos de padrón y titular), RegistroAcceso (detalle de movimiento auditado), EstadoVehicular (estado circunstancial) y tipos enumerados (TipoMovimiento: ENTRADA/SALIDA, PuertaAcceso, TipoPersona).
• **Lógica de Negocio (AccesoVehicularModel.kt):** Implementa las reglas estrictas de seguridad institucional: (1) Formato mínimo y normalización de placas, (2) Detección de coherencia de permanencia (si está dentro solo sale, si está fuera solo entra), (3) Validación y marcado de auditoría para excepciones forzadas, (4) Tratamiento de conductores ocasionales preservando el titular en padrón, y (5) Persistencia de transacciones.
¿Por qué va acá? (Justificación Técnica):
Las reglas de negocio pertenecen al **negocio de la universidad**, no a la interfaz de usuario ni a los controladores. El Modelo no sabe ni le interesa si la aplicación se muestra en un teléfono Android, en una página web o en una pantalla de consola. Si la universidad cambia sus normativas de acceso, solo se modifica el Modelo sin tocar la vista ni el controlador, garantizando el Principio de Responsabilidad Única (SRP).

4.2. Componente: VISTA (View) — Interfaz y Presentación Pasiva

¿Qué contiene?
• **Contrato de Vista (IVigilanciaView.kt):** Interfaz que define los métodos abstractos de renderizado: mostrarEstadoVehicular(), mostrarConfirmacionExito(), mostrarMensajeError(), mostrarHistorial() y mostrarCargando().
• **Implementación Concreta (ConsolaGaritaView.kt / Pantalla Android Compose):** Código que dibuja en pantalla y captura los clics del usuario.
¿Por qué va acá? (Justificación Técnica):
La Vista debe ser estrictamente una **Vista Pasiva (Passive View)**. Su única labor es desplegar los datos que recibe y transmitir los eventos del usuario hacia el controlador. No debe contener sentencias SQL, ni validar restricciones de seguridad, ni decidir qué pantalla abrir. Gracias al contrato abstracto IVigilanciaView, podemos cambiar la interfaz por completo sin alterar el Modelo.

4.3. Componente: CONTROLADOR (Controller) — Orquestación y Flujo

¿Qué contiene?

• La clase VigilanciaController.kt, que enlaza la abstracción IVigilanciaView con el AccesoVehicularModel.

• Manejadores de eventos: onPlacaDetectadaOIngresada(), onRegistrarAccesoSolicitado() y onConsultarHistorialSolicitado().

¿Por qué va acá? (Justificación Técnica):

El Controlador es el **mediador que desacopla la Vista del Modelo**. Cuando el vigilante interactúa o el sensor OCR detecta una placa, el evento llega al Controlador; este le solicita al Modelo que procese los datos, evalúa el resultado estructurado y decide a qué método de la Vista instruir para actualizar lo que el usuario ve.

5. Modelado Visual y Diagramas de Ingeniería del Software

5.1. Diagrama de Casos de Uso del Escenario

El diagrama de casos de uso modela los requerimientos funcionales y los servicios que el sistema ofrece en el puesto de control de garita:

• **Oficial de Vigilancia (Actor Primario):** Personal en garita que interactúa con la interfaz para ingresar placas, validar datos y confirmar ingresos o egresos.
• **Cámara IA / OCR (Sensor Externo):** Alimenta de forma automatizada el caso de uso *CU01: Capturar / Ingresar Placa*.
• **Base de Datos SQLite (Almacén de Datos):** Asienta los registros generados en *CU03*, *CU04* y alimenta las consultas de *CU07*.

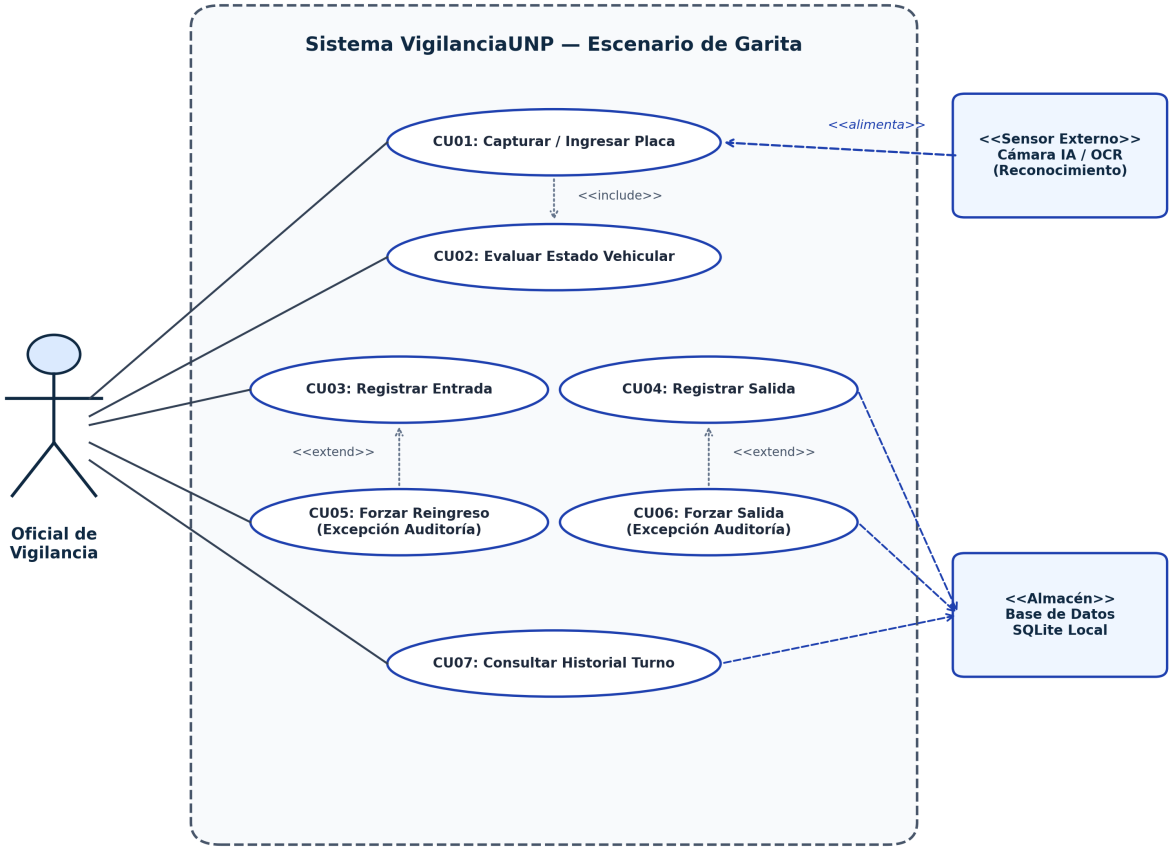


Figura 1: *Diagrama de Casos de Uso UML — Escenario: Control y Registro en Garita (VigilanciaUNP).*

5.2. Diagrama de Actividades del Proceso de Garita

El diagrama de actividades detalla el flujo procedimental ordenado en calles (*swimlanes*) entre los tres actores/capas:

1. **Calle 1 (Vigilante / Dispositivo):** Captura de placa (manual o por visión artificial), visualización de los datos del titular y confirmación de la orden de registro.
2. **Calle 2 (Controlador):** Reenvío estructurado de eventos, recepción de estados y despacho de mensajes de confirmación visual hacia la interfaz.
3. **Calle 3 (Modelo & SQLite):** Ejecución de la lógica de evaluación (¿vehículo figura dentro o fuera?), bifurcación de permisos (Salida regular vs. Entrada regular) y asentamiento con marca temporal.

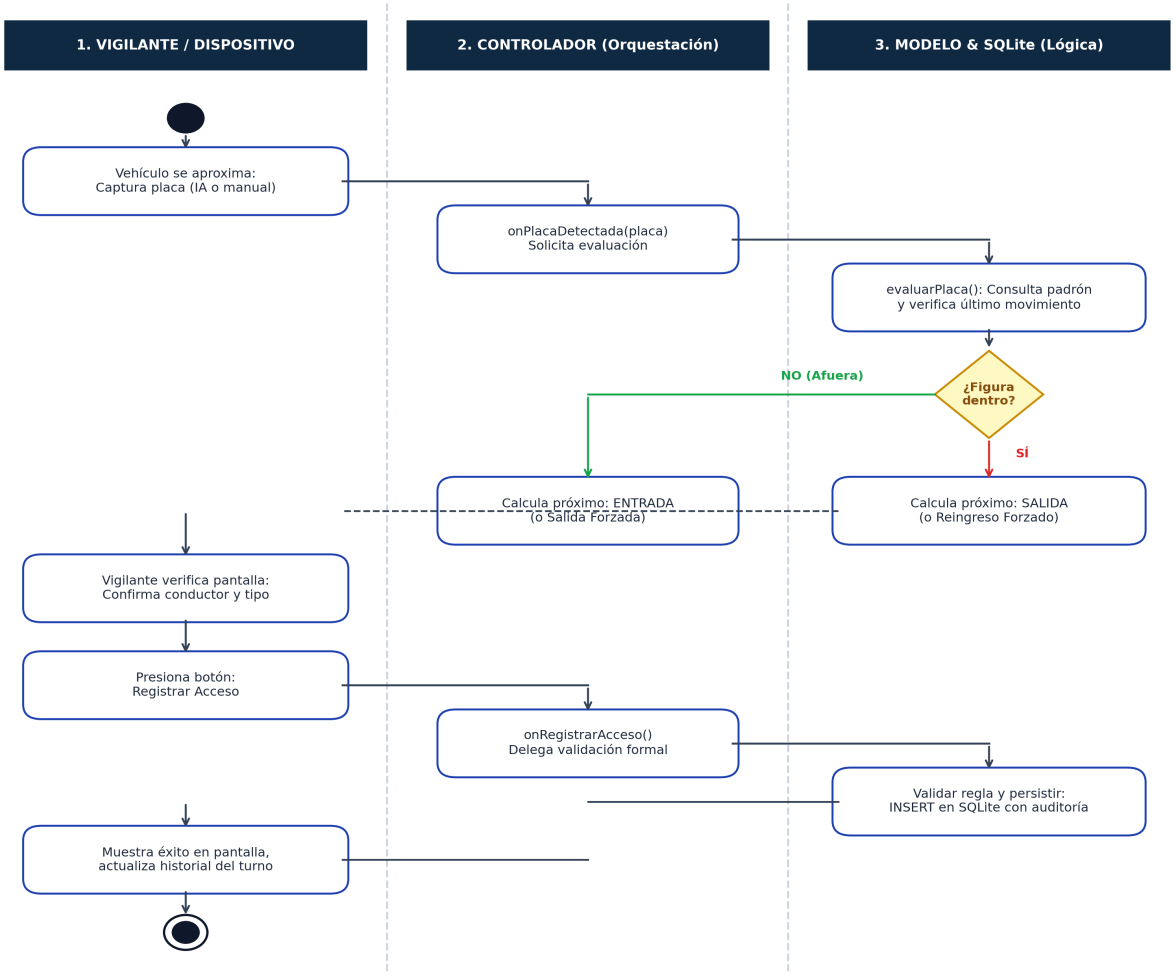


Figura 2: *Diagrama de Actividades UML con Swimlanes — Flujo Operativo y Decisiones en Garita.*

5.3. Diagrama de Clases: Arquitectura Modelo-Vista-Controlador

El diagrama de clases muestra la descomposición formal en los paquetes modelo, vista y controlador:

- **Paquete vista:** Contiene la interfaz abstracta IVigilanciaView y la implementación concreta ConsolaGaritaView.
- **Paquete controlador:** Aloja VigilanciaController, que depende de la abstracción IVigilanciaView y del modelo.
- **Paquete modelo:** Reúne el gestor de lógica AccesoVehicularModel y las entidades inmutables Vehiculo y RegistroAcceso.

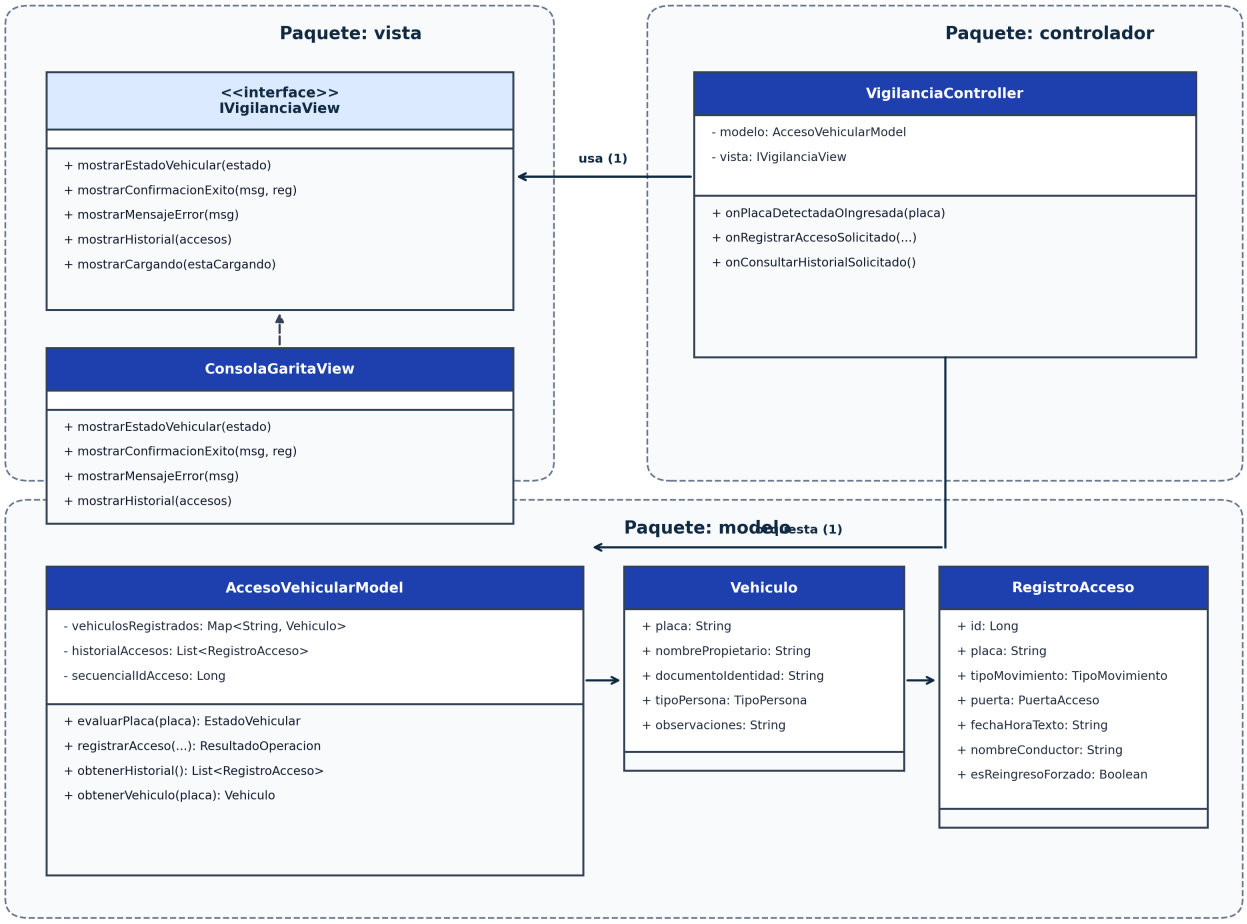


Figura 3: *Diagrama de Clases UML — Estructura Estática y Desacoplamiento del Patrón MVC.*

5.4. Diagrama de Secuencia de Interacción en Garita

El diagrama de secuencia modela el intercambio cronológico de mensajes durante una transacción de acceso vehicular completa:

• **Fase de Consulta (Mensajes 1 al 9):** El vigilante ingresa la placa. La vista delega al controlador, este consulta evaluarPlaca() al modelo; el modelo consulta antecedentes en SQLite y retorna el EstadoVehicular, permitiendo a la vista habilitar el botón de acción correspondiente.
• **Fase de Registro (Mensajes 10 al 17):** Al presionar 'Registrar ENTRADA', el controlador delega a registrarAcceso() en el modelo; este verifica la coherencia, asienta el registro en SQLite (#ID generado) y devuelve un resultado exitoso para confirmación visual en la garita.

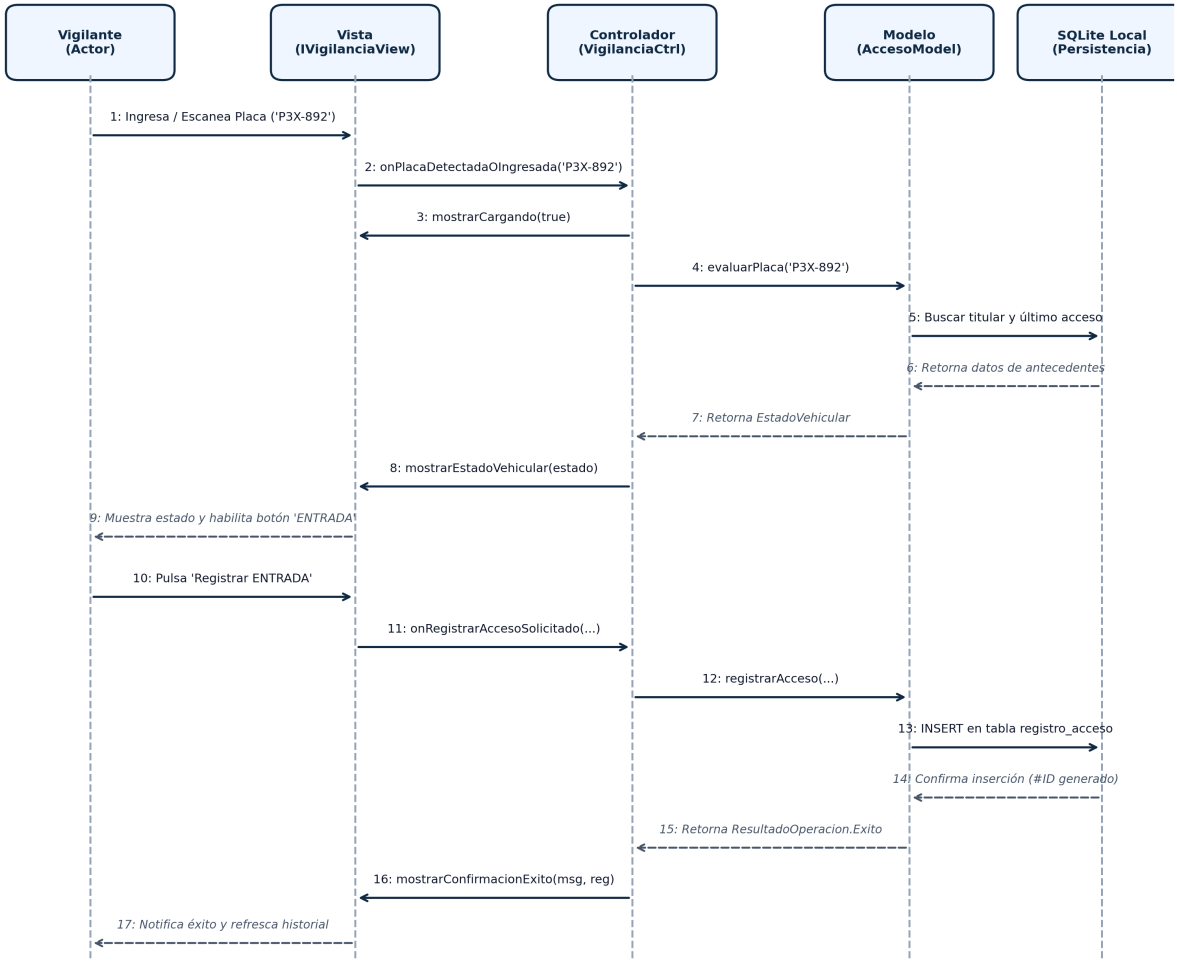


Figura 4: *Diagrama de Secuencia UML — Ciclo de Mensajes de una Petición de Acceso en Garita.*

6. Conclusiones y Beneficios de la Arquitectura

1. Mantenibilidad y Separación de Intereses (SoC): Al aislar la lógica de negocio en el *Modelo*, las normativas de seguridad de la UNP (como restricciones horarias o validaciones de carnets) pueden modificarse sin tocar la interfaz gráfica ni el controlador.
2. Flexibilidad de Presentación: Gracias al desacoplamiento mediante la interfaz IVigilanciaView, el sistema puede desplegarse en una aplicación móvil nativa con Jetpack Compose, en una pantalla web de garita o en terminales embebidas sin alterar el núcleo del sistema.
3. Auditoría y Robustez Institucional: La inclusión de marcas explícitas de auditoría (reingresos y salidas forzadas, conductores ocasionales) garantiza trazabilidad completa ante incidentes de seguridad dentro del campus universitario.
4. Facilidad de Pruebas Unitarias: El Modelo y el Controlador pueden someterse a pruebas automatizadas independientes en cuestión de milisegundos, sin necesidad de compilar interfaces gráficas pesadas ni emuladores móviles.